Energy-Efficient Code via LLMs

Group 18

June 2026

Contents

1 Introduction & Context

1.1 Context & Background

Over the past few years, software engineering has undergone a drastic change in how software is produced and how quickly it is produced. One of the main factors that contributed to this drastic change is the emergence of Large Language Models (LLMs). Large Language Models were at first seen only as a tool that software developers used to replace traditional ways of learning like engaging in online communities (e.g., StackOverflow) [2], that would end up taking hours of their time. Nowadays, they became actual coding assistants that sometimes generate a large part of the code that goes into production, making software a lot more efficient to produce [3], as software engineers only need to review the code that would have otherwise be written by themselves in a much longer time. Despite all these advantages, as concerns about sustainability emerge across numerous fields, it is important to examine whether code generated by LLMs is energy-efficient and sustainable in practice.

1.2 Problem Statement

Although a substantial body of research has evaluated code generated by LLMs, most studies focus primarily on correctness. Assessing correctness is undoubtedly essential when determining whether LLMs can be reliably used for software development. However, correctness alone is not sufficient. We must also examine the energy efficiency of LLM-generated solutions and determine whether these models can produce code that is not only functionally correct and suitable for deployment in production environments, but also meets the requirements of energy-efficient software.

2 Research Questions & Methodology

In this section, we analyze each of the paper's research questions (RQs), contextualize and explain the relevance of each question, and detail the steps and design choices the authors implemented to answer them. We also propose a non-trivial alternative method that the authors could have used, and we explain why choosing between the original and this alternative represents a real decision dilemma. To support our alternative, we cite existing literature.

2.1 Luca Ratan - Research Question

2.1.A Isolated Research Question

"How does the energy efficiency of LLM-generated code vary when using different LLMs?" ([1] - RQ_0)

2.1.B Articulation and Relevance

Our main objective is to find out how energy efficient the code written by LLMs is, by comparing LLM-generated code with human-generated code and code generated by a Green software expert. However, it is important to also do a comparison between the LLM models themselves, before transitioning to comparison with a human or a Green software expert. Answering this question will help us with setting the context of the experiment, to see if there is any energy consumption difference between the code generated by different models. This helps with seeing if the LLMs can be seen as a general, singular subject for comparison - meaning that all the models produce the same results in terms of energy consumption -, or if each of them produces different results, because after all, each LLM model is structured and trained differently.

From a **practical** standpoint, this is important especially for the developers community, which nowadays, are increasingly using LLMs for writing code not just as a tool, but as an assistant. We also aim to compare the energy efficiency of the LLM-generated code on multiple hardware platforms: a server, a PC, and a Raspberry Pi. Based on the results we will have, a developer can make an optimal choice in terms of energy efficiency when using an LLM as a coding assistant, depending on the hardware context in which they are working.

From a **theoretical** standpoint, we want to cover a less explored attribute for evaluating the quality of LLM-generated code. So far, evaluations of code generated this way consisted of only checking the correctness, i.e., the code produces the right output for a given problem. With the analysis of energy consumption, we can now think of a new way to evaluate LLMs. Moreover, we will also obtain a better perspective over how different models, having different architectures, training methods, training datasets, and number of parameteres perform with respect to this evluation of energy efficiency. Maybe none of them were specifically trained with the purpose of generating energy-efficient code in mind, but somehow, based on the different properties mentioned earlier, they might have "inherited" this capacity.

2.1.C Methodological Deconstruction

We aim to select the top 6 accessible, unique LLMs in the difficult category of the EvoVal leaderboard, as of July 2024, based on two main criteria:

- **Accessbility:** the model is either open-source of available through API.
- **Uniqueness:** the model uses a different base model compared to the other already-chosen LLMs.

This will imply choosing LLMs created by different companies, such as OpenAI (ChatGPT), DeepSeek (DeepSeek), Meta (Codellama), or BudEcosystem (CodeMillenials)

The EvoVal leadeboard was built on the HumanEval benchmark by OpenAI and includes 828 coding problems across different domains on which LLMs are evaluated based on the capacity of generating solutions for those problems [4]. The reasons for choosin EvoVal as a source for the LLMs are that it is one of the most recent benchmarks for evaluating LLMs trained for code generation, and contains multiple test cases, and also a source-of-truth solution for each problem.

This way of choice will give us a certain level of external validity, as we will have a larger variety of models in terms of architecture, sizes, and training method.

Moving forward to the coding problem selection, we want to select 100 coding problems from the Difficult category in EvoVal, since we want to generate Python code with higher computational complexity and additional requirements compared to the other categories. We will only keep the problems for which each of the chosen LLMs generated a correct solution, since correctness will need to be already validated. We will evaluate these solutions based on the test cases we will extract from EvoVal. These initial solutions that we will test in order to see if they will provide the correct results will be the "functional solutions" representing the solutions that we will extract based on the original prompt given to each LLM, for each problem, from EvoVal.

These original prompts that will create the functional solutions will also be used to create different types of prompts. As this is not highly relevant for this specific research question, we won't dive deeper into what these different prompts represent. If a solution generated from an LLM, for a specific prompt, proves not to be correct after 6 reprompts, that specific

solution will be removed from the analysis. Once again, the solutions will be verified against the extracted test cases from EvoVal to see if they will be correct. All the remaining solutions will be executed on three hardware platforms that we chose, with the purpose of strengthening the external validity by offering results accross a wide spectrum of hardware specifications.

We will calculate a number X_p^m (m - hardware machine, p - problem) of iterations with the purpose of achieving a runtime of 3 minutes. On each machine m, all test cases will be executed for each variant of a solution to problem p for X_p^m iterations. Each of these runs will be repeated 21 times on each hardware platform.

2.1.D Alternative Methodology Contrast

- **The Dilemma:** A fundamentally different approach could consist of different choices in the following aspects of the study:
  1. **LLM Selection**: choose more LLMs that vary in more different ways, not just architecture or parameters. The variety could consist of availability (open-source or closed-source) or of LLMs chosen specifically from big tech companies like Amazon, Google, or Meta. This would generalize the comparison and results even more.
  2. **Problem Selection**: increase the variety once again in this aspect of the study by choosing significantly more problems that vary not only in algorithmic categories,but also in difficulty.
  3. **Hardware Platform Selection**: only choose one hardware platform.
  4. **Energy Measurement**: instead of using tools like EnergiBridge and Monsoon, use a more accessible tool, like the `perf` Linux tool.

  The dilemma of choosing between the two methodologies is based on multiple trade-offs. Firstly, the new methodology would provide far greated external validity, as results can be questioned much easier when they are based upon a small number of LLMs or problems.

  Moreover, the original methodology is far stricter, as the final problems for the analysis are carefully selected based on whether or not each LLM generated a correct solution for them, while in this new methodology, a regeneration cycle is applied in which the LLMs are reprompted up to 25 times, and the first passing solution is considered for the analysis. This can cause unfair comparisons, as some LLMs that might manage to generate a correct solution after very few reprompts will be compared to LLMs that might have needed a lot more attempts. The first case represents an “innate” capacity of the LLM to generate a correct solution, while the second case represents more of a “learning” capacity. On the other hand, this option also allows for a much larger variety of problems to be included in the analysis. The paper’s methodology might result in only the easier problems from the chosen difficult category to be selected, as those problems give higher chances to LLMs to successfully generate a correct solution on the first try. Besides this, it also doesn’t represent a realistic scenario, as in the real world, developers usually keep reprompting the LLM until they get a satisfactory solution.

  Another trade-off between the two methodologies is represented by the precision of the analysis. Apsan et al. [1] (the paper in question) used 21 repetitions per run, combined with tools like EnergiBridge and Monsoon, which resulted in high statistical accuracy. This new methodology includes only 5 repetitions, using the `perf` Linux tool for the energy consumption analysis, making the execution easier to reproduce by other researchers.

With regard to the hardware platform selection, the original methodology would be more insightful in terms of how dependable energy efficiency is on hardware platforms, providing researchers and developers with a clearer option for choosing the right LLM for the right hardware. When it comes to the new methodology, the results would provide a more general perspective, which could still be useful for most developers who simply use a company laptop, with the advantage of having a simpler experiment design by disregarding a confounding factor.

- **Literature Support:** Literature support can be found at [5], in which the authors gather multiple open-source and closed-source LLMs from different companies, like Mistral, Google, Meta, or Amazon, and compare all the LLMs with each other, and also with canonical human-written solutions, in terms of the energy efficiency of the code that is produced by them. In the study, the LLMs are prompted to generate solutions for 878 programming problems that vary in difficulty and algorithmic categories, gathered from the world known platform Leetcode.

2.2 Hugo Hulsebosch - Research Question

2.2.A Isolated Research Question

"What is the difference between human code and LLM-generated code in terms of energy-efficiency?" ([1] - RQ_1)

2.2.B Articulation and Relevance

In software creation, tools that use LLMs to generate code are used more often by new and experienced developers. These tools are increasingly replacing code that would have been human-written. Therefore, the relevant question is whether this shift improves or worsens energy consumption. Human-written code is the logical place of comparison, as it represents what LLM-generated code is replacing in practice.

Before this paper, research was mainly focused on the performance, readability, and maintainability of LLM-generated code. Energy efficiency has received (far less) attention in prior research, while software sustainability and Green AI are becoming more important in research and industry. If LLM-generated code is systematically less energy-efficient than code by human developers, then widespread adoption of tools using these LLMs leads to large-scale energy waste. Before being able to investigate RQ_2 and RQ_3, we must first understand how LLM-generated code performs relative to human-written code (RQ_1), as a baseline. Answering this question shows whether a gap in energy efficiency exists between human-written and LLM-generated code. If such a gap exists, it shows that efforts should be made to improve the energy efficiency of LLM-generated code. If not, then the industry can rely on LLMs without concerns about energy waste.

2.2.C Methodological Deconstruction

The authors use a quantitative, empirical research design structured as a controlled experiment. The study design follows guidelines for empirical software engineering [6], [7] and energy efficiency assessment [8], [9].

The independent variable is the origin of the code. This can be either human-written or generated by an LLM. The dependent variable is the energy consumption used by the solution. The energy consumption can differ between machines. Therefore, experiments are conducted over various hardware: a server, a pc, and a Raspberry Pi.

The human baseline comprises the canonical solutions in the EvoEval benchmark. The functional solutions are the LLM-generated solutions. These solutions are generated by six selected LLMs, using the original EvoEval prompt, without any prompt modification related to energy efficiency.

Each solution variant is executed 21 times on each platform to factor out random occurrences. The EnergiBridge measuring software is used to measure energy consumption on the server and PC. For the Raspberry Pi, this is Monsoon Power Monitor.

2.2.D Alternative Methodology Contrast

- **The Dilemma:** The canonical solutions from EvoEval are only a single implementation per problem: the details of the writer and the environment in which it was produced are unknown. The human side of comparison is therefore small and maybe unrepresentative. An alternative is to recruit multiple developers and let them independently solve the same problem. Varying in experience, coding style, and optimization skills to generalize more. The downside is that recruiting is very time-consuming (as Justus Bogner stated in his guest lecture), adds confounding factors (program language proficiency differences, energy-efficiency awareness), which are avoided by canonical solutions. Dilemma: canonical solutions have higher reproducibility (you can take the exact same solutions in a retry) but lower representativeness. Recruiting developers reflects real-world conditions better but has lower experimental control.
- **Literature Support:** A recent study using the alternative method is Wang et al. [10], who used this approach in comparing different LLMs on software engineering tasks. They recruited 109 developers with at least one year of professional developer experience to independently solve coding tasks. Moreover, a foundational study by Prechelt [11] uses a similar style. In this study, 80 programmers implemented the same problem independently. An important finding from this paper is that the variation between programmers in the same language was often bigger than variation between languages. This shows that a single canonical solution from EvoEval might not represent how different programmers approach the same problem.

2.3 Silvio Bolognani - Research Question

2.3.A Isolated Research Question

"What is the impact of prompt engineering techniques on the energy efficiency of LLM-generated code?" ([1] - RQ_2)

2.3.B Articulation and Relevance

- **Relevance:** Prompt engineering is an emerging field that tries to determine the optimal prompt structure to obtain LLM outputs tailored to certain requirements. At a deeper level, the question is trying to establish whether it is worth it to invest time and resources in prompt engineering in an energy-saving context.
- **Results:** From the results, we can surmise that while prompt engineering does have an effect on the code generated by the LLM, this change in output code does not necessarily imply an improvement in energy efficiency, especially when the machine running the code varies. This presents quite a challenge, as a developer would need to know a priori what hardware the software would run on. Depending on the problem at hand, this might not be possible. The findings evident from this study also seem to agree with the limited theory that is slowly being established.

2.3.C Methodological Deconstruction

- **Research structure:** The research question and the following process follow a tried and true plan, in line with the established computer science research pipeline. The inductive approach applied by the researchers is common in the software engineering field and is well-suited for a niche and emerging field such as prompt engineering, which does not have much established theory. The experiment choice is also well suited, as running in vivo experiments involving real-life-like code bases would be outside the scope of a single paper.
- **Reproducibility:** Regarding the reproducibility of the experiment, the author does a good job of describing the setup used, both in hardware and testing framework. The author does provide executables, which can be used to replicate the energy usage measurements. Some of the LLMs used are queried through API and are thus susceptible to architecture changes and updates which makes recreating results impossible. Another issue with running benchmarks with LLMs is their inherently non-deterministic behaviour; the code generated by prompting is not necessarily going to be the same at each prompt run. Multiple runs can be executed, and statistics can be extracted from the obtained distribution, but the high variance between outputs makes the experiment less valuable.

2.3.D Alternative Methodology Contrast

- **The Dilemma:** While the experimental design is sound and follows in the steps of several other projects also trying to gauge the efficiency and performance of LLM-generated code [12]. An alternative way this question could be approached is by simulating more real-life-like scenarios, reducing the problem set to a couple of generic categories comprising entire systems that fulfill real-life needs, instead of less complex problems. The programs would then be run on the various hardware systems (when appropriate). Energy measurements could then be taken, like in the existing paper or at a more granular level, such as in [13], where each component of the system is individually monitored, thus giving insight into how different LLMs (and LLMs in general) prioritize efficiency of different resources. To achieve this, the prompting strategy should also be changed. As system program complexity grows, multiple prompts could be needed to adjust and fix issues with the implementation. Because of this, a significant part of the project would revolve around the prompt engineering stage. Understandably, in the context of the overall paper, this design would have been hard to implement and would have required significant changes to the rest of the research questions. This different methodology would allow for a more detailed understanding of how an LLM can handle efficient resource management across complex systems, which better represent commonly occurring workloads.
- **Literature Support:** The paper at [12] also tries to answer a similar question to the research paper being analyzed in this review, with the main difference being in the experimental setup, namely the energy measurements being taken at the resource level (i.e., individual measurements for CPU, memory). The other paper at [13] investigates the performance of LLM-generated code but puts more emphasis on iterative improvements through prompt engineering, and correctness of the generated code.

2.4 Gabriel Marica - Research Question

2.4.A Isolated Research Question

"What is the difference between code developed by a Green software expert and LLM-generated code in terms of energy efficiency?" ([1] - RQ_3)

2.4.B Articulation and Relevance

As more developers rely on AI tools to produce code faster, the volume of code that is generated and deployed is also likely to increase. This makes it important to evaluate code not only for functional correctness, but also for non-functional qualities such as energy efficiency. In many cases, developers focus primarily on functionality and may overlook sustainability concerns. Therefore, it is valuable to understand how LLM-generated code compares with code written by a Green software expert in terms of energy efficiency, since this can have significant implications for the sustainability of software applications and the environment.

2.4.C Methodological Deconstruction

The authors utilized an Empirical Style, such that they first setup a controlled experimental environment to execute code across different hardware platforms. Following standard methodological definitions (Bhattacherjee, 2012 [14]), the proposition claims a relationship bewtween two constructs: code and energy efficiency. To test this proposition, the hypothesis is that the expert-written code consumes less energy than the LLM-generated code. The moderating variable is represented by three different hardware platforms: server, laptop, and Raspberry Pi. They selected a set 9 coding problems from the EvoEval benchmark, motivated by [4], but focused on specifically 4 of these for the Green software expert and LLM-generated code comparison. For AI generation, they used 6 different LLMs combined with 4 distinct prompting strategies. In regards to the Green software expert, a specialist with over 7 years of experience manually optimized the code using the problem description, the canonical solutions, test cases and a set of specially curated guidelines. Finally, to measure the dependent variable, defined as energy consumption in Joules, the researchers executed each solution for approximately 3 minutes. They repeated these measurements 21 times on each hardware platform to account for intrinsic energy fluctuations and to strengthen the statistical power.

2.4.D Alternative Methodology Contrast

- **The Dilemma:** An alternative method the authors could have used is a Taxonomical Style, where they could have focused on the structural analysis of the source code generated by both the Green software expert and the LLMs. So, instead of setting up an experimental environment to measure energy consumption, the code could be evaluated based on structural features that are known to influence energy efficiency, such as data structures, algorithmic complexity, and design patterns. The fundamental trade-off between the original method and this alternative is that the physical execution provide a direct measurement of energy consumption, which is a more concrete and empirical approach. In contrast, the taxonomical method relies on theoretical analysis. By classifying and labeling the code based on its structural features, the risk is that it may not capture the real-world energy consumption accurately.
- **Literature Support:** Literature support can be found at [15]. In this paper, a static approach for code analysis is used. The researchers chose this method because it is lightweight and provides developers with quick and highly detailed estimates, down to a single line of source code. This strategy avoids the need for expensive physical power monitors or slow software simulators. The analysis time is fast and it can be integrated directly into a developer's IDE.

2.5 Daniele Di Cesare - Research Question

2.5.A Isolated Research Question

To what extent does the hardware platform[1] impacts the energy efficiency of code generated by LLMs? See footnotes[2].

2.5.B Articulation and Relevance

When developers increasingly rely on large language models (LLMs) to generate software, there is often an implicit assumption that the resulting code will exhibit consistent performance characteristics across execution environments. However, this assumption is challenged by the fact that different hardware architectures, ranging from high-performance servers to consumer-grade CPUs and resource-constrained IoT devices, exhibit fundamentally different instruction execution patterns, caching behaviors, and energy profiles. Prior work in energy-aware and "Green AI" computing has demonstrated that computational efficiency is highly dependent on both model design and hardware context [16]. Despite this, relatively little attention has been given to how these hardware-dependent effects manifest in code generated by LLMs, particularly in terms of portability of energy efficiency across heterogeneous systems.

The importance of this research question lies in the rapidly growing deployment of AI-generated code in production environments on a global scale. As billions of lines of machine-generated code are integrated into cloud services, enterprise systems, and edge devices, even small inefficiencies can aggregate into substantial financial and environmental costs. A key unresolved gap in the literature is whether energy-efficient code produced or validated in one computational context (e.g., a developer workstation or optimized server environment) retains its efficiency when executed on fundamentally different hardware platforms, such as distributed servers or low-power IoT devices. While existing research has extensively studied algorithmic efficiency and hardware-specific optimization separately, there is a lack of empirical understanding of cross-architecture generalization of energy efficiency in LLM-generated code. Addressing this gap is critical to ensure that the benefits of automated code generation do not lead to increased energy consumption and environmental impact in real-world deployments.

2.5.C Methodological Deconstruction

To answer this specific RQ, the researchers ran all tests on three physical machines with very different hardware specs. The machines were a high-end server, a PC, and a Raspberry Pi. To make sure they were measuring only the code's energy and not background noise, they turned off all non-essential background processes, set the CPU power governor to performance, and set a cooldown period of 60 seconds between tests so the machines wouldn't overheat and alter results. They executed each piece of code 21 times per machine (discarding the first run to avoid cold starts). Because the scripts run very quickly, they wrapped the functions in loops and subtracted the energy of an "empty pass" function to isolate the code's precise consumption. They recorded energy consumption using EnergiBridge for a total of approximately 4.6 billion energy measures. The authors ran a statistical test called an Two-Way Aligned Rank Transformation (ART) ANOVA. This specific test allowed them to see if the interaction between the Model used and the Hardware platform was statistically

[1] Server, PC, and Raspberry Pi

[2] Even tho this is not an explicit research question of the paper, the impact of the hardware platform is a recurring theme of this research.

significant. The researchers also tried to nudge LLMs into a more appropriate solution for the platform by telling the AI the hardware profile the code was going to be executed on, but as RQ_2 shows, prompt engineering does not have a significant impact on energy consumption, sometimes even obtaining worse results.

2.5.D Alternative Methodology Contrast

- **The Dilemma:** Instead of measuring physical power usage on real machines, the authors could have used profiling tools like `perf` to track micro-architectural metrics like cache misses and branch mispredictions. These micro-architectural metrics could also be obtained by running the LLM-generated code inside specialized software simulators (like GEM5 [17]) that support different CPU instruction set architectures (e.g., x86 or ARM) and configurable memory. Testing on real hardware gives you actual, authentic physical data reflecting real-world usage. However, it suffers from a lot of unpredictable hardware noise. It is also incredibly slow and expensive to test and does not generalize well to other hardware. On the other hand, the proposed alternative completely isolates the software. It can tell you exactly why a piece of code is draining power by tracking microscopic variables (e.g., "this nested loop caused 5,000 L1 cache misses on ARM but only 200 on x86"). It is 100% repeatable with zero environmental noise. The downside is that the results are software approximations, and they do not capture the chaotic nature of real-world systems.
- **Literature Support:** [17] describes gem5 as a highly configurable, cycle-accurate system simulator that enables controlled evaluation of microarchitectural behavior across different ISAs and memory hierarchies, supporting reproducible analysis of performance-related metrics. [16] highlights that existing studies on sustainable code generation using LLMs largely focus on functional correctness and developer productivity, while energy efficiency and cross-environment execution behavior remain underexplored and lack standardized evaluation approaches.

3 Conclusion & Reference List

3.1 Conclusion

In this research, we looked at the energy efficiency of code generated by LLMs. We broke this down into five research questions, addressing comparisons with human-written code, the impact of prompting strategies, the role of Green software expertise, and the influence of different hardware platforms. Our findings show that while LLMs are good at writing code that works, writing code that's energy-efficient is still an unsolved problem. Energy consumption varies a lot between models, and human-written code, particularly Green software experts, still has a clear sustainability edge. Prompt engineering techniques meant to help were often inconsistent in practice. One key insight is that energy efficiency is tied to where the code actually runs. Code optimized for a desktop PC can behave very differently on a server or a small device like a Raspberry Pi, sometimes using far more power than expected. Ultimately, this research suggests that the software engineering community needs to think beyond development speed if we want a more sustainable digital future.

3.2 Reference List

[1] R. Apsan, V. Stoico, M. Albonico, R. Dhar, K. Vaidhyanathan, and I. Malavolta, "Generating Energy-Efficient Code via Large-Language Models – Where are we now?." [Online]. Available: https://arxiv.org/abs/2509.10099

[2] G. Burtch, D. Lee, and Z. Chen, "The consequences of generative AI for online knowledge communities," *Scientific Reports*, vol. 14, p. , 2024, doi: 10.1038/s41598-024-61221-0.

[3] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot." [Online]. Available: https://arxiv.org/abs/2302.06590

[4] C. S. Xia, Y. Deng, and L. Zhang, "Top Leaderboard Ranking = Top Coding Proficiency, Always? EvoEval: Evolving Coding Benchmarks via LLM." [Online]. Available: https://arxiv.org/abs/2403.19114

[5] M. A. Islam *et al.*, "Evaluating the Energy-Efficiency of the Code Generated by LLMs." [Online]. Available: https://arxiv.org/abs/2505.20324

[6] F. Shull, J. Singer, D. I. K. Sjøberg, and S. (Online Service, *Guide to Advanced Empirical Software Engineering*. London: Springer London, 2008.

[7] C. Wohlin, P. Runeson, HöstMartin, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in software engineering*. Berlin: Springer Berlin, 2014.

[8] A. Guldner *et al.*, "Development and evaluation of a reference measurement model for assessing the resource and energy efficiency of software products and components—Green Software Measurement Model (GSMM)," *Future Generation Computer Systems*, vol. 155, pp. 402–418, 2024, doi: https://doi.org/10.1016/j.future.2024.01.033.

[9] J. Mancebo, F. García, and C. Calero, "A process for analysing the energy efficiency of software," *Information and Software Technology*, vol. 134, p. 106560, June 2021, doi: https://doi.org/10.1016/j.infsof.2021.106560.

[10] W. Wang, H. Ning, G. Zhang, L. Liu, and Y. Wang, "Rocks Coding, Not Development–A Human-Centric, Experimental Evaluation of LLM-Supported SE Tasks." [Online]. Available: https://arxiv.org/abs/2402.05650

[11] L. Prechelt, "An Empirical Comparison of Seven Programming Languages," *Computer*, vol. 33, no. 10, pp. 23–29, 2000, doi: 10.1109/2.876288.

[12] L. Lacher, M. Lewis, E. Ruetschle, and A. Sickafoose, "Performance of LLM-Generated Code." May 2026.

[13] "SSRN Paper 6810423." [Online]. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6810423

[14] A. Bhattacherjee, *Social Science Research: Principles, Methods, and Practices*. 2012. [Online]. Available: http://scholarcommons.usf.edu/oa_textbooks/3

[15] S. Hao, D. Li, W. G. J. Halfond, and R. Govindan, "Estimating mobile application energy consumption using program analysis," in *2013 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 92–101. doi: 10.1109/ICSE.2013.6606555.

[16] S. B. M. Ali, O. Kirmani, A. Hameed, S. M. Danish, and G. Srivastava, "Sustainable Code Generation Using Large Language Models: A Systematic Literature Review." [Online]. Available: https://arxiv.org/abs/2603.00989

[17] J. Lowe-Power *et al.*, "The gem5 Simulator: Version 20.0+." [Online]. Available: https://arxiv.org/abs/2007.03152

[18] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining GitHub," in *Proceedings of the 11th Working Conference on Mining Software Repositories*, Hyderabad India: ACM, May 2014, pp. 92–101. doi: 10.1145/2597073.2597074.